软件详细设计文档

1. 文档概述

1.1 编写目的

本文档基于《软件需求规格说明书》《软件体系结构设计文档》，对"面向 CPU+XPU 异构系统的智能调度算法及中间件原型"的设计元素进行精化和细化，把体系结构中的"黑盒子"子系统转变为"白盒子"的类与构件，达到程序员可直接据此编写 C++ 代码的程度。本文档同时纳入《用户界面设计文档》中 `schedctl` 客户端的设计结果。

1.2 读者对象

项目开发团队成员（上官源博、张豫君等）、指导教师、评审人员。

1.3 术语定义

术语	说明
DQN	深度 Q 网络，调度决策模型
ONNX / ONNX Runtime	模型交换格式 / 跨平台推理引擎
UDS	Unix Domain Socket，本机进程间通信
HAL	硬件抽象层
procfs / sysfs / devfreq	Linux 内核暴露状态与调频能力的文件接口
状态快照	监控线程写入、决策线程读取的共享内存结构
契约	Python 训练侧与 C++ 运行侧之间的输入/输出维度与归一化约定

1.4 参考资料

- 《软件需求规格说明书.md》
- 《软件体系结构设计文档.md》
- 《用户界面设计文档.md》
- 项目申请书《面向大模型推理任务的边缘异构平台智能调度算法研究与实现》
- ONNX Runtime C++ API 文档；Linux kernel procfs / devfreq 文档

2. 系统概述

2.1 系统目标

构建可集成于 openEuler 的轻量化调度中间件守护进程 `schedd`，通过 DQN 推理为 LLM 推理任务动态分配 CPU/NPU 资源、批处理大小与频率档位，并由管理人员通过 `schedctl` 完成运维。

2.2 主要功能

请求推理调度、采集系统状态、调整处理器频率、查询系统状态、启停服务、切换调度模式（六项）。

2.3 目标用户

推理引擎（程序）、管理人员（运维）、操作系统（外部系统）。

2.4 运行环境

openEuler 24.03+（aarch64，RK3588）；开发期 WSL + openEuler（x86_64）仿真；C++17 + CMake；ONNX Runtime；systemd 托管。

3. 设计目标和原则

3.1 设计目标

- 每个需求用例都有明确的类协作实现方案，且可追踪到需求条目。
- 关键路径（请求 → 决策 → 执行）实现细化到方法级（签名、前/后置条件、算法）。
- 屏蔽平台差异：x86 仿真与 aarch64 实机共用同一套设计类，仅替换设备实现。
- 与既有代码资产对齐：`sched::SystemFS`、`sched::INpu`、`monitor::Monitor` 等已落地类作为设计基准。

3.2 设计原则

原则	落实方式
高内聚低耦合	每类单一职责；跨子系统只经接口类（`ServiceInterface`/`SystemInterface`）交互
信息隐藏	属性默认 private，仅接口方法 public；设备差异收敛在 `INpu` 之下
依赖倒置	上层依赖抽象 `INpu` 而非 `RealNpu`/`VirtualNpu`
契约优先	Python/C++ 之间的输入维度与归一化先冻结为 `model_contract.json`
可测试性	监控与 HAL 可脱离守护进程被单元测试；设备经工厂注入 mock

4. 用例设计

4.1 分析类到设计类的转化

分析类按体系结构设计元素落位，转化方式遵循 sdd 的三种方式：

分析类	类型	设计类	转化方式
`serviceInterface`	boundary	`service::ServiceInterface`	方式一（职责单方法实现）
`askInterface`	boundary	`service::ServiceInterface` + `service::RequestDispatcher`	方式三（协作完成：监听与路由分离）
`schedManager`	control	`agent::SchedManager`	方式一
`decisionMaker`	control	`agent::DecisionMaker` + `FeatureEncoder` + `ActionDecoder`	方式三（编码/推理/解码协作）

分析类	类型	设计类	转化方式
decisionExecutor	control	agent::DecisionExecutor	方式一
systemMonitor	control	agent::SystemMonitor	方式一（对 System 的门面）
OSMonitor	boundary	system::OsMonitor + monitor::Monitor + sched::SystemFS	方式三（读取/计算/文件访问协作）
ScheduleInterface	entity	system::SystemInterface	方式一
SetInterface	entity	system::SystemManager::applyAction	方式二（一组方法实现）
FreqSetter	boundary	system::SystemManager + sched::INpu::SetFrequency	方式二

体系结构 HAL 抽象补充出下列设计类（无对应分析类，属实现层细化）：sched::INpu、sched::VirtualNpu、sched::RealNpu、agent::FeatureEncoder、agent::ActionDecoder、common::StateSnapshot。

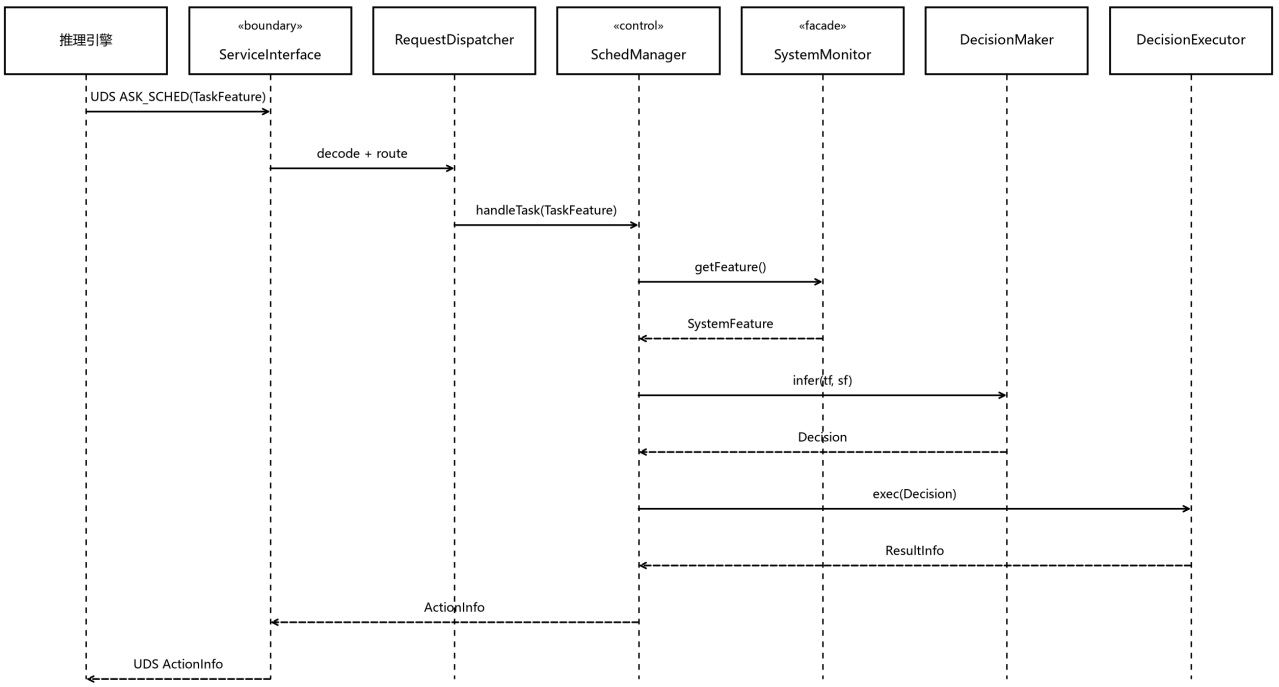
4.2 UC1 请求推理调度（askSched）

实现方案：推理引擎经 UDS 提交任务特征，Service 子系统解析后交给 Agent 的 SchedManager；SchedManager 依次索取系统状态（UC2）、执行 DQN 推理、下发动作（UC3），最终回传 ActionInfo。

步	设计类协作	说明
1	ServiceInterface::onRequest	epoll 收到可读事件，读入完整报文
2	RequestCodec::decode	反序列化为 TaskFeature
3	RequestDispatcher::route	按报文类型路由到 SchedManager::handleTask
4	SchedManager::handleTask	调 SystemMonitor::getFeature()（展开为 UC2）
5	DecisionMaker::infer	FeatureEncoder 编码 → ONNX Run → ActionDecoder 解码
6	DecisionExecutor::exec	经 SystemMonitor::applyAction() 下发（展开为 UC3）
7	RequestCodec::encode	ActionInfo 序列化后经 UDS 回写

后置条件：推理引擎收到与请求一一对应的 ActionInfo；Agent 内部状态（模式）不变。

用例设计顺序图：



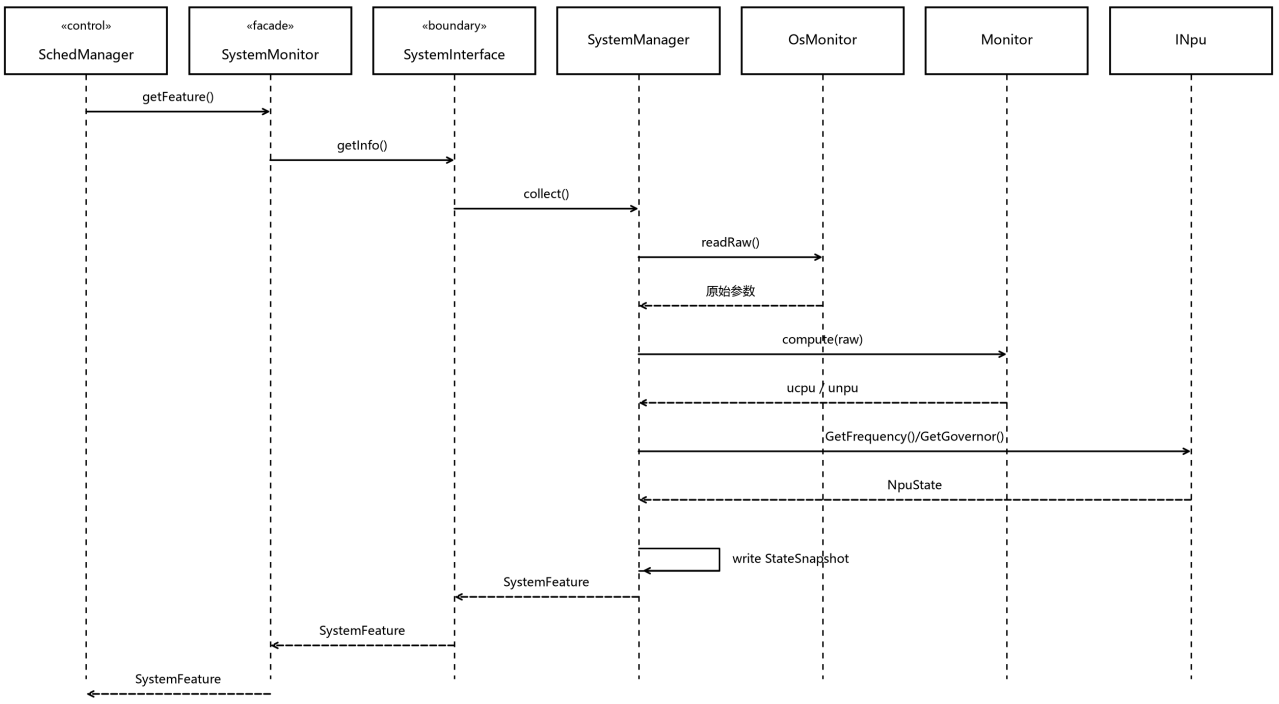
4.3 UC2 采集系统状态（SystemFeature）

实现方案：`SchedManager` 通过门面 `SystemMonitor` 获取状态；`SystemMonitor` 经 `SystemInterface` 委派给 `SystemManager`；`SystemManager` 调 `OsMonitor`/`monitor::Monitor`/`sched::INpu` 读取原始参数并换算为状态量。

步	设计类协作	说明
1	`SchedManager` → `SystemMonitor::getFeature()`	请求系统特征
2	`SystemMonitor` → `SystemInterface::getInfo()`	门面转发（子系统间唯一通道）
3	`SystemInterface` → `SystemManager::collect()`	子系统内编排
4	`SystemManager` → `monitor::Monitor::get_ucpu()`	两次采样 `/proc/stat` 求差
5	`SystemManager` → `sched::INpu::GetLoad()`	取 NPU 三核负载/频率/governor
6	`SystemManager` → `StateSnapshot` 写入共享内存	供决策线程零拷贝读取
7	逐级返回 `SystemFeature`	上游使用

异常流：设备文件不可读 → 返回上一快照并标记 `stale=true`，不阻塞调度。

用例设计顺序图：



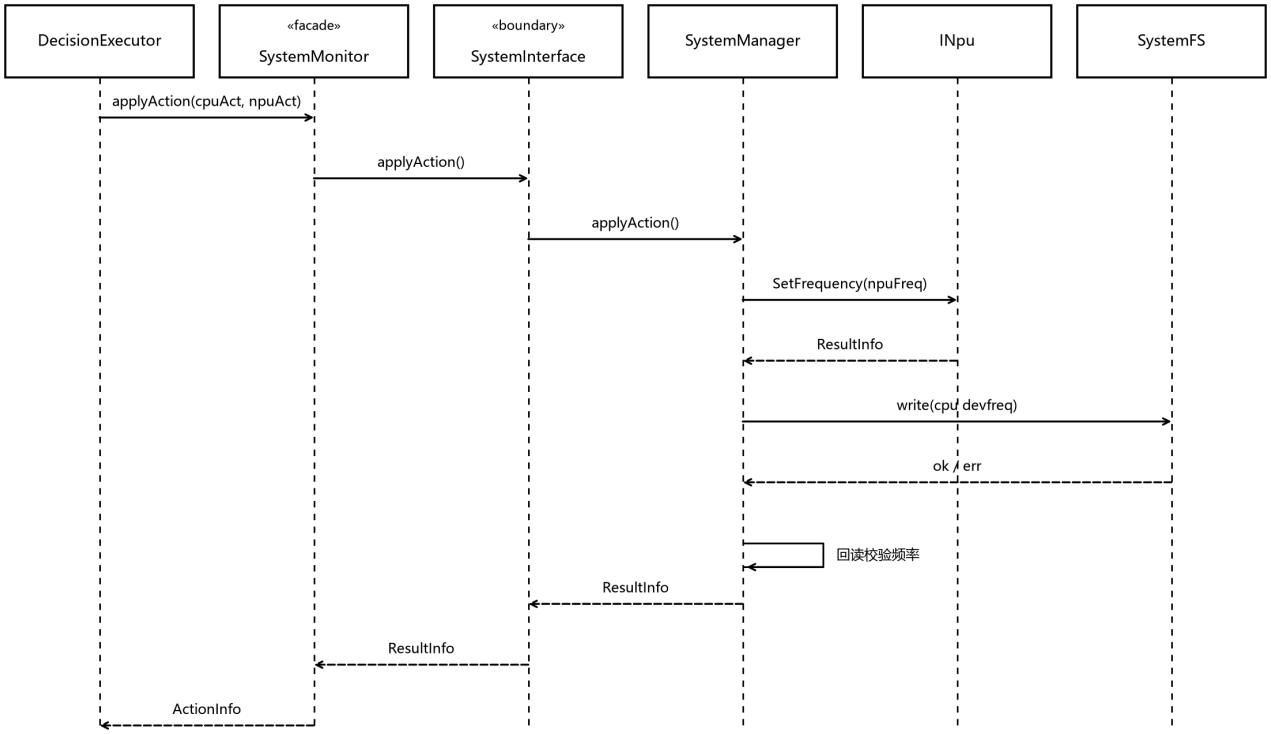
4.4 UC3 调整处理器频率（changeFreq）

实现方案：`DecisionExecutor` 将 `Decision` 拆为 CPU 与 NPU 两组动作，经 `SystemMonitor` → `SystemInterface` → `SystemManager`，最终由 `sched::INpu::SetFrequency` / CPU devfreq 写入落地。

步	设计类协作	说明
1	`DecisionExecutor::exec(dec)`	拆解决策为 CPU/NPU 动作
2	`SystemMonitor::applyAction(cpu_act)`	门面转发
3	`SystemInterface::applyAction`	子系统入口
4	`SystemManager::applyAction`	校验频率落在可用档位（devfreq `available_frequencies`）
5	`sched::INpu::SetFrequency` / `SystemFS::write`	写入 NPU / CPU 频率
6	回读校验	读取生效频率，组装 `ResultInfo`
7	返回 `ActionInfo`	含逐设备结果

异常流：频率越界 → 夹取到最近合法档位并置 `adjusted=true`；写入失败 → `ResultInfo.ok=false`，由 `SchedManager` 触发降级。

用例设计顺序图：



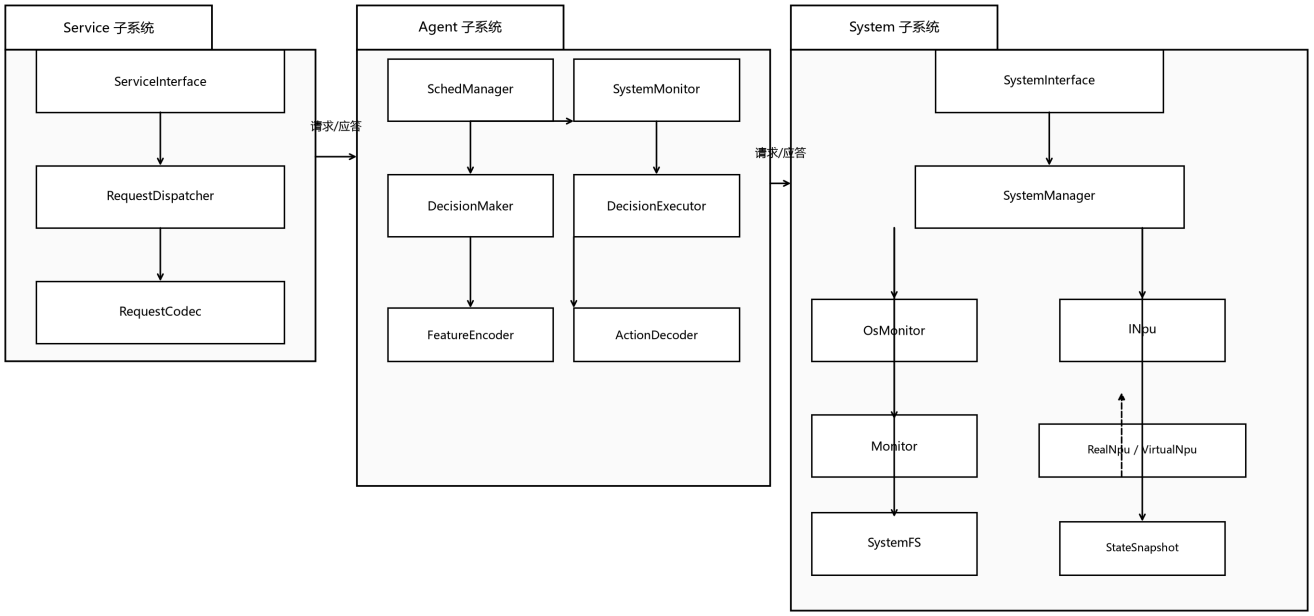
4.5 UC4 / UC5 / UC6 管理类用例

用例	实现方案
UC4 查询系统状态	`schedctl status` → `StatusCommand` → `CtlClient` 经 UDS 发 `GET_STATE` → `ServiceInterface` → `SystemMonitor::getFeature()` → 原路返回并格式化
UC5 启停服务	`schedctl service <动作>` → `ServiceCommand` → `SystemdCtl` 调 `systemctl <动作> schedd`；守护进程生命周期由 systemd 管理，不进入 `schedd` 内部
UC6 切换调度模式	`schedctl mode <模式>` → `ModeCommand` → `CtlClient` 发 `SET_MODE` → `ServiceInterface` → `SchedManager::setMode()`；模式非法或降级不可用时返回错误

说明：UC4~UC6 的界面侧设计见《用户界面设计文档》，此处只给出与设计类的对接点。

4.6 设计类图（用例设计视图）

用例设计产生的设计类及其协作关系如下（整体类图见第 6 章）：



5. 子系统 / 构件设计

> 经与用户确认，本阶段**执行**。本项目预计规模中等（约 20~30 个类、3 个构件），子系统边界需要明确以支撑并行开发与独立测试。

5.1 子系统划分与职责

子系统	对应层次	职责	对外接口
Service	服务接口层	与推理引擎及 `schedctl` 双向交互	`ServiceInterface`（UDS 协议）
Agent	调度决策层	推理调度过程控制、模式管理	`SchedManager`（进程内调用）
System	状态监管层 + 硬件服务层	系统状态采集与设备参数访问	`SystemInterface`（进程内调用）
CLI（独立构件）	—	管理人员命令行客户端	复用 Service 的 UDS 协议

5.2 Service 子系统

职责：唯一对外通信入口，负责协议解析、请求路由、应答回写、连接生命周期。

内部类

类	职责
service::ServiceInterface	创建/绑定/监听 UDS，epoll 事件循环，accept/读写
service::RequestDispatcher	按报文类型路由到处理器（调度/查询/模式）
service::RequestCodec	报文编解码（长度前缀 + 二进制头 + JSON 载荷）

对外接口：UDS 套接字路径 /run/schedd.sock；报文类型 ASK_SCHED / GET_STATE / SET_MODE。

交互：RequestDispatcher 将 ASK_SCHED 交给 agent::SchedManager，GET_STATE 交给 agent::SystemMonitor，SET_MODE 交给 SchedManager::setMode。

5.3 Agent 子系统

职责：实现调度决策流程与运行模式，是"调度大脑"，不含设备细节。

内部类

类	职责
agent::SchedManager	流程总控；持有门面与推理器；管理模式状态机
agent::SystemMonitor	对 System 子系统的门面，Agent 内统一访问入口
agent::DecisionMaker	ONNX Runtime 封装，加载模型并执行推理
agent::FeatureEncoder	任务特征 + 系统特征 → 模型输入向量（归一化）
agent::ActionDecoder	模型 Q 值 → 离散动作 → Decision
agent::DecisionExecutor	决策落地，拆解为设备动作并校验结果

对外接口：SchedManager::handleTask、SchedManager::setMode、SystemMonitor::getFeature。

5.4 System 子系统

职责：把设备参数转换为决策可用状态量，并把调整命令落地到设备。

内部类

类	职责
system::SystemInterface	子系统对外唯一入口
system::SystemManager	参数↔状态换算、档位校验、状态快照维护
system::OsMonitor	读取 procfs/sysfs 原始参数
monitor::Monitor	CPU/NPU 利用率与负载计算（既有代码资产）
sched::SystemFS	跨平台文件读写封装（既有代码资产）
sched::INpu（抽象）	NPU 能力抽象（既有代码资产）
sched::RealNpu / sched::VirtualNpu	真实 / 虚拟 NPU 实现（既有代码资产）

类	职责
`common::StateSnapshot`	共享内存状态快照结构

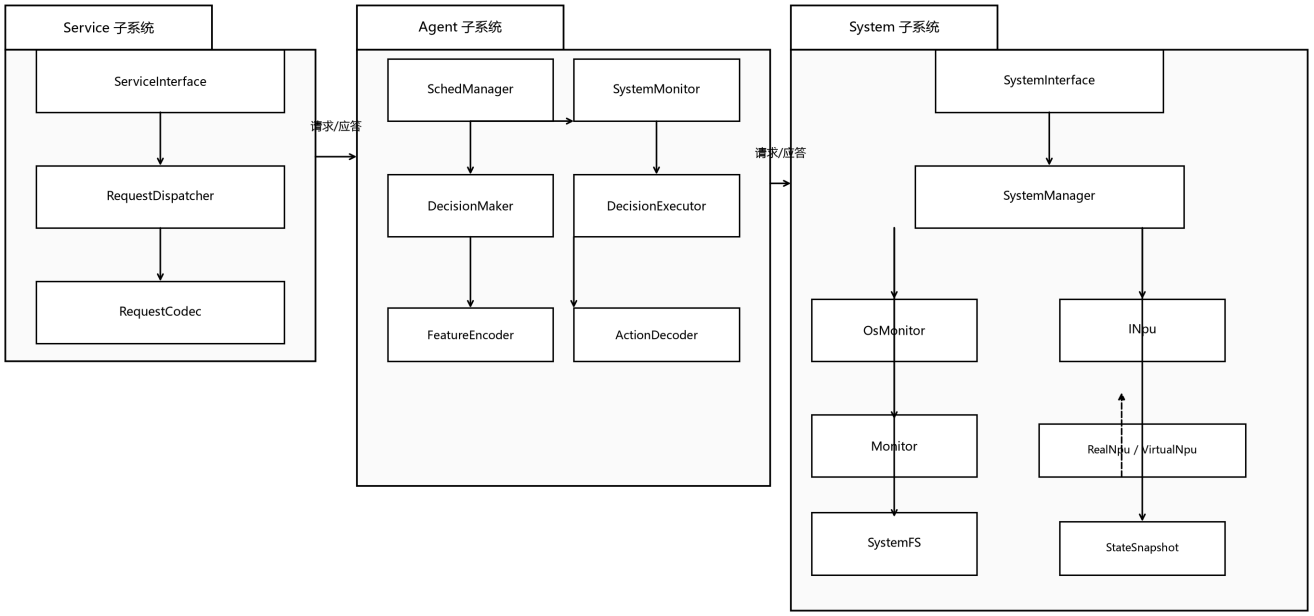
对外接口：`getInfo()`、`applyAction()`。

5.5 构件与构建（CMake）

构件（target）	类型	内容	依赖
`sched_tool`	静态库	`SystemFS`、`INpu` 及实现	无
`sched_monitor`	静态库	`monitor::Monitor`、`NpuState`	`sched_tool`
`sched_system`	静态库	System 子系统	`sched_tool`、`sched_monitor`
`sched_agent`	静态库	Agent 子系统	`sched_system`、onnxruntime
`sched_service`	静态库	Service 子系统	`sched_agent`
`schedd`	可执行	守护进程主编排、线程启动	全部
`schedctl`	可执行	CLI 客户端	`sched_common`
`*_test`	可执行	各子系统的单元/冒烟测试	对应库

6. 类设计

6.1 设计类图



6.2 类清单与可见性

设计类	子系统	类可见性	说明
`ServiceInterface`	Service	public	进程外可见（UDS）
`RequestDispatcher`	Service	internal	仅 Service 内部
`RequestCodec`	Service	internal	仅 Service 内部
`SchedManager`	Agent	public	Agent 对外
`SystemMonitor`	Agent	public	Agent 对 System 的门面
`DecisionMaker`	Agent	internal	仅 Agent 内部
`FeatureEncoder` / `ActionDecoder`	Agent	internal	仅 Agent 内部
`DecisionExecutor`	Agent	internal	仅 Agent 内部
`SystemInterface`	System	public	System 对外
`SystemManager`	System	internal	仅 System 内部
`OsMonitor`	System	internal	仅 System 内部
`INpu` / `RealNpu` / `VirtualNpu`	System(tool)	public	HAL 抽象
`SystemFS`	System(tool)	public	文件封装
`Monitor`	System(monitor)	public	监控计算
`StateSnapshot`	common	public	共享数据结构

6.3 类属性与方法详表

仅列关键类；属性默认 private，方法标注作用范围。

`agent::SchedManager`

成员	类型	说明
`m_monitor`	`std::shared_ptr<SystemMonitor>`	门面，不可空
`m_maker`	`std::shared_ptr<DecisionMaker>`	推理器
`m_executor`	`std::shared_ptr<DecisionExecutor>`	动作执行器
`m_mode`	`SchedMode`	当前模式（auto/static/off）
`m_modeMutex`	`std::shared_mutex`	模式读写保护

方法	签名	前置	后置	算法
`handleTask`	`ActionInfo handleTask(const TaskFeature&)`	`m_mode != off`	返回非空 ActionInfo	取状态→推理→执行→组装

方法	签名	前置	后置	算法
setMode	bool setMode(SchedMode)	模式取值合法	m_mode 更新	加写锁更新；off→static 需模型可用
mode	SchedMode mode() const	—	—	加读锁读取

agent::DecisionMaker

成员	类型	说明
m_env / m_session	Ort::Env / Ort::Session	ONNX Runtime 会话
m_encoder / m_decoder	FeatureEncoder / ActionDecoder	编码/解码
m_timeout	std::chrono::microseconds	推理超时（默认 5ms）
m_ready	bool	模型是否可用

方法	签名	前置	后置	算法
load	bool load(const std::string&, const std::string& contract)	路径可读	m_ready=true	建会话，按契约校验输入维度
infer	std::optional<Decision> infer(const TaskFeature&, const SystemFeature&)	m_ready	超时返回 nullopt	编码→session.Run→解码 argmax
ready	bool ready() const	—	—	返回 m_ready

system::SystemManager

成员	类型	说明
m_monitor	monitor::Monitor	CPU/NPU 计算
m_cpu / m_npu	std::shared_ptr<sched::INpu>	设备抽象
m_fs	sched::SystemFS	文件访问
m_snap	common::StateSnapshot*	共享内存指针
m_availFreqs	std::vector<int>	可用频率档位（启动时读取）

方法	签名	前置	后置	算法
collect	SystemFeature collect()	设备可读	返回状态量	采样→换算→写快照
applyAction	ResultInfo applyAction(const DeviceAction&)	频率档位已加载	设备频率已变更	夹取档位→写入→回读校验

common::StateSnapshot（共享内存布局，POD）

字段	类型	说明
`seq`	`std::atomic<uint64_t>`	版本号（写前后各 +1，双写检测）
`ucpu`	`float`	CPU 利用率
`unpu[3]`	`float[3]`	NPU 三核负载
`cpuFreq` / `npuFreq`	`int`	当前频率
`mode`	`int`	当前调度模式
`tsNs`	`int64_t`	采样时间戳

6.4 类间关系

源	关系	目标	重数
`SchedManager`	组合	`SystemMonitor`	1:1
`SchedManager`	组合	`DecisionMaker`	1:1
`SchedManager`	组合	`DecisionExecutor`	1:1
`DecisionMaker`	组合	`FeatureEncoder` / `ActionDecoder`	1:1
`SystemMonitor`	依赖	`SystemInterface`	1:1
`SystemManager`	聚合	`INpu`	1:2（CPU/NPU）
`SystemManager`	组合	`Monitor` / `SystemFS`	1:1
`RealNpu` / `VirtualNpu`	实现	`INpu`	—
`SystemManager`	依赖	`StateSnapshot`	1:1

6.5 状态图（`SchedManager` 模式状态机）

```
[*] --> auto
auto --> static : 模型不可用 / 推理连续超时
static --> auto : 模型恢复且显式切回
auto --> off  : 管理人员 mode off（二次确认）
static --> off: 管理人员 mode off
off --> static: 管理人员 mode static
off --> auto  : 管理人员 mode auto（模型可用）
```

约束（后置条件）：`off` 模式下 `handleTask` 不再触发推理与调频，仅返回当前状态摘要；`static` 模式使用固定规则策略（不可用 DQN 时的降级路径），满足可靠性需求。

7. 数据设计

7.1 需要持久保存的数据

中间件为**瞬时状态 + 点对点通信**系统，无业务数据库需求。需要持久保存的仅为**配置**与**模型资产**；运行状态（`StateSnapshot`）为易失的共享内存，不落盘。

数据	持久性	载体
调度配置（模式、档位上下限、推理超时、socket 路径）	持久	配置文件 `schedd.conf`
DQN 模型	持久	`dqn_model.onnx`
模型契约（输入维度、归一化范围、动作映射）	持久	`model_contract.json`
实时状态快照	易失	POSIX 共享内存 `/dev/shm/schedd_state`
运行日志	追加	journald（经 `sd_journal` / stderr）

不引入数据库：单机单进程、无并发写、无查询需求，引入 SQLite/关系库成本高于收益（与体系结构文档一致）。

7.2 配置文件设计（`/etc/schedd/schedd.conf`）

采用 INI 风格（由 `common::ConfigLoader` 解析）：

节	键	类型	默认	说明
`[server]`	`socket_path`	string	`/run/schedd.sock`	UDS 路径
	`max_conn`	int	`16`	最大并发连接
`[agent]`	`default_mode`	enum	`auto`	启动模式
	`infer_timeout_us`	int	`5000`	推理超时（微秒）
	`model_path`	string	`/usr/lib/schedd/dqn_model.onnx`	模型路径
	`contract_path`	string	`/usr/lib/schedd/model_contract.json`	契约路径
`[system]`	`cpu_freq_min` / `cpu_freq_max`	int	硬件读取	频率夹取范围
	`npu_freq_min` / `npu_freq_max`	int	硬件读取	频率夹取范围
	`sample_interval_ms`	int	`500`	采样间隔

7.3 模型契约设计（`model_contract.json`）

字段	说明
`input_dim`	输入向量维度（如 8）

字段	说明
`output_dim`	动作数（A）
`state_bounds`	每个输入分量的归一化上下界
`actions`	动作编号 → 语义映射（CPU 份额、批大小、CPU/NPU 档位）

契约的 `input_dim`/`state_bounds`/`actions` 必须在 Python 训练侧与 C++ 侧一致，`DecisionMaker::load` 据此校验，避免维度错配。

7.4 运行时共享内存结构

见 6.3 中 `common::StateSnapshot`。写入采用"双写序号（seqlock）"：写前 `seq++`（变奇数），写完 `seq++`（变偶数）；读方读取前后序号一致才认为快照有效，实现无锁零拷贝。

7.5 数据操作

操作	触发者	说明
配置读取	启动时 `ConfigLoader::load`	只读一次，缺失键取默认值
模型加载	启动时 `DecisionMaker::load`	失败则进入 `static` 降级
快照写	监控线程 `SystemManager::collect`	每 `sample_interval_ms` 一次
快照读	决策线程 `SchedManager::handleTask`	读前校验 seqlock
频率写	`SystemManager::applyAction`	写后回读校验
日志写	各子系统	结构化字段经 journald

8. 设计模式应用

模式	类型	应用点	理由
单例（Singleton）	创建型	守护进程主对象 `ScheddApp`	进程内唯一编排者，避免多重初始化
工厂方法（Factory Method）	创建型	`sched::INpu::Create()`	由环境变量 `SCHED_NPU_MODE` 决定返回 `RealNpu`/`VirtualNpu`，屏蔽平台差异
外观 / 门面（Facade）	结构型	`agent::SystemMonitor`	为 Agent 提供访问 System 的单一入口，降低跨子系统耦合
适配器（Adapter）	结构型	`RealNpu` / `VirtualNpu` 实现 `INpu`	统一真实设备与虚拟设备的访问接口

模式	类型	应用点	理由
策略（Strategy）	行为型	调度模式 auto/static/off	不同模式对应不同决策策略，可运行时切换
命令（Command）	行为型	`schedctl` 的 `Command` 派生类	将每类运维操作封装为对象，便于扩展与统一分发
代理（Proxy）	结构型	模式切换时的模式代理	非 auto 模式下拦截推理请求，避免无效推理

9. 评审意见

#	评审方面	评审要点	结论
①	规范性	文档结构、模型表达是否规范	通过：按 8 章结构组织，类图/顺序图/状态图表达规范
②	简练性	语言是否简洁易懂	通过：采用表格与签名清单，避免冗余叙述
③	正确性	是否正确实现功能与非功能需求	通过：六项用例均有实现方案；毫秒级决策、降级与自愈由超时/模式机制保证
④	可实施性	是否细化到可编码、可基于选定技术实现	通过：方法级签名、前/后置条件、算法齐备；与既有 `tool/`、`monitor/` 代码对齐
⑤	可追踪性	需求是否都有对应设计条目	通过：4.1 给出分析类→设计类映射，4.2~4.5 覆盖全部六用例
⑥	一致性	模型之间、图文之间是否一致	通过：类清单、属性方法表与类图一致；术语与需求/体系结构文档统一
⑦	高质量	是否考虑有效性、可靠性、可扩展性、可修改性	通过：seqlock 降延迟、降级策略保可靠、INpu 抽象利扩展、门面与接口降耦合

评审结论：七项全部通过。详细设计可作为编码实现依据。

9.1 遗留风险

风险	影响	应对
DQN 收敛性不足（avg reward 偏弱）	推理决策质量	编码期间继续训练；`static` 规则调度作为保底
契约冻结前 Python/C++ 不通联调	集成返工	先落地 `model_contract.json` 并做数值对齐测试

风险	影响	应对
真实 NPU 12 月才可用	上板验证推迟	x86 用 `VirtualNpu` 全链路仿真，接口不变